

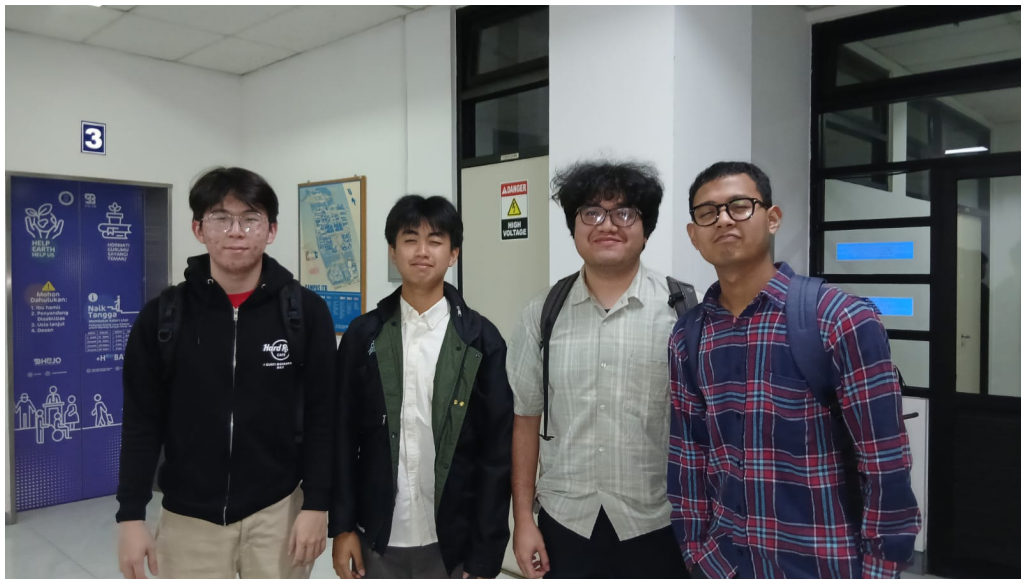
Tugas Besar

Milestone 2

IF2224 Teori Bahasa Formal dan Otomata

Lexical Analysis

Semester 2 Tahun 2025/2026



Disusun oleh:

Kai

Jonathan Kris Wicaksono	(13524023)
Vincent Rionarlie	(13524031)
Muhammad Aufar Rizqi Kusuma	(13524061)
Bryan Pratama Putra Hendra	(13524067)

Laboratorium Ilmu dan Rekayasa Komputasi
Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	4
1.3	Tujuan	4
1.4	Batasan Masalah	5
2	Dasar Teori	6
2.1	Teori Singkat	6
2.1.1	Parser dan Syntax Analysis	6
2.1.2	Parse Tree	7
2.1.3	Recursive Descent Parser	9
3	Perancangan dan Implementasi	11
3.1	Gambaran Umum Program	11
3.2	Algoritma Parser	11
3.3	Struktur Direktori Program	12
3.4	Implementasi Recursive Descent Parser	13
3.4.1	Alur Utama Parsing	13
3.4.2	Pemilihan Produksi Dengan Lookahead	15
3.4.3	Struktur Fungsi Per Non-Terminal	15
3.4.4	Hierarki Ekspresi dan Prioritas Operator	16
3.4.5	Variabel Dengan Komponen Berantai	17
3.4.6	Penanganan Error dan Validasi Token	18
3.5	Implementasi Kode	19
3.5.1	main.cpp	19
3.5.2	lexer/token.h	19
3.5.3	lexer/token.cpp	20
3.5.4	lexer/token_stream_reader.h	20
3.5.5	lexer/token_stream_reader.cpp	21
3.5.6	parser/parser.h	22
3.5.7	parser/parser.cpp	22
3.5.8	parser/parse_tree_node.h	24
3.5.9	parser/parse_tree_node.cpp	25
3.5.10	parser/parse_tree_utils.h	25
3.5.11	parser/parse_tree_utils.cpp	26



4	Pengujian	27
4.1	Kasus Dasar	27
4.2	<i>Edge Cases</i>	33
5	Penutup	39
5.1	Kesimpulan	39
5.2	Saran	39
6	Lampiran	40
6.1	Tautan GitHub	40
6.2	Tabel Kontribusi	40

Bab 1

Pendahuluan

1.1 Latar Belakang

Compiler adalah perangkat lunak yang menerjemahkan source code dari bahasa tingkat tinggi menjadi representasi yang dapat dieksekusi oleh mesin. Keberadaan compiler penting karena memungkinkan programmer menulis program dengan struktur dan ekspresi yang mendekati cara manusia berpikir, namun tetap menghasilkan instruksi yang dapat dipahami oleh komputer. Dalam praktik rekayasa perangkat lunak, compiler tidak hanya berfungsi sebagai penerjemah, tetapi juga sebagai penjaga konsistensi aturan bahasa, sehingga program yang dihasilkan bersifat benar dan dapat dijalankan.

Proses kompilasi terdiri dari beberapa tahap yang saling bergantung. Tahap lexical analysis memecah karakter menjadi token; syntax analysis memeriksa apakah urutan token membentuk struktur yang sah; semantic analysis memvalidasi makna, tipe data, dan aturan konteks; sedangkan code generation mengubah representasi internal menjadi bentuk target. Setiap tahap memiliki peran yang kritis karena kesalahan pada tahap awal akan menghambat atau merusak proses pada tahap berikutnya. Oleh sebab itu, pembangunan compiler yang benar menuntut setiap tahap dikerjakan secara sistematis.

Pada mata kuliah IF2224 Teori Bahasa Formal dan Otomata, tugas besar Arion Compiler dirancang untuk menghubungkan teori dengan implementasi nyata. Bahasa Arion yang bersifat Pascal-like dipilih agar mahasiswa dapat menerapkan konsep grammar formal, otomata, serta teknik parsing pada bahasa yang memiliki struktur jelas. Pengerjaan bertahap melalui milestone mendorong pemahaman yang mendalam atas keterkaitan antar komponen compiler, sekaligus memberi ruang untuk menguji hasil secara terpisah pada setiap fase.

Milestone 1 telah menyelesaikan pembangunan lexer yang menghasilkan daftar token dari source code Arion. Token inilah yang menjadi masukan utama bagi Milestone 2, yaitu pembangunan parser atau syntax analyzer. Parser bertugas memeriksa struktur sintaksis program dan memastikan token-token tersebut mengikuti grammar yang telah ditentukan, serta membangun parse tree sebagai representasi hierarkisnya. Dengan demikian, Milestone 2 menjadi jembatan antara tokenisasi dan analisis semantik yang akan dilaksanakan pada milestone berikutnya.

Pembangunan parser pada milestone ini sangat terkait dengan teori Context-Free

Grammar (CFG) dan teknik Recursive Descent. CFG mendefinisikan struktur bahasa dalam bentuk produksi non-terminal ke terminal dan non-terminal, sedangkan Recursive Descent merealisasikan setiap non-terminal sebagai fungsi rekursif dalam kode C/C++. Keduanya memberikan pendekatan yang langsung dan transparan dalam mengimplementasikan parser, sehingga proses analisis sintaks dapat ditelusuri sesuai dengan kaidah formal yang dipelajari.

1.2 Rumusan Masalah

Penyusunan parser untuk Arion Compiler memunculkan kebutuhan untuk merumuskan pertanyaan penelitian yang menjembatani teori dan implementasi. Laporan ini berangkat dari pertanyaan utama mengenai bagaimana membangun parser yang mampu menganalisis struktur sintaksis dari daftar token hasil lexer sehingga sesuai dengan grammar bahasa Arion. Pertanyaan ini mencakup bagaimana token-token disusun ulang menjadi konstruksi program yang valid dan bagaimana aturan grammar ditegakkan secara konsisten.

Secara lebih teknis, tantangan berikutnya adalah bagaimana mengimplementasikan algoritma Recursive Descent agar setiap produksi grammar dapat direpresentasikan dengan jelas dalam kode C/C++. Implementasi ini harus memperhatikan urutan produksi, pemilihan cabang grammar berdasarkan token lookahead, serta pemetaan non-terminal menjadi fungsi-fungsi parser yang terstruktur dan mudah dilacak.

Selain itu, laporan ini juga mengkaji bagaimana membangun dan menampilkan parse tree yang menggambarkan hierarki struktur program Arion. Parse tree tidak hanya berfungsi sebagai keluaran parser, tetapi juga sebagai jembatan menuju tahap semantic analysis pada milestone selanjutnya. Oleh karena itu, representasi parse tree harus akurat, lengkap, dan konsisten terhadap grammar.

Aspek lain yang tidak kalah penting adalah bagaimana parser menangani kesalahan sintaksis secara informatif. Parser harus mampu mendeteksi token yang tidak sesuai, menunjukkan lokasi kesalahan, serta memberikan pesan yang jelas agar pengguna dapat memperbaiki program. Pertanyaan mengenai strategi error handling ini menjadi bagian penting dalam memastikan parser bersifat robust dan dapat digunakan pada pengujian nyata.

1.3 Tujuan

Tujuan utama Milestone 2 adalah membangun komponen parser Arion Compiler yang dapat menganalisis daftar token dari Milestone 1 menggunakan algoritma Recursive Descent. Parser yang dibangun harus mampu memvalidasi struktur program secara sintaktis dan menghasilkan parse tree sebagai keluaran utama. Pencapaian ini menjadi fondasi agar pipeline kompilasi dapat berlanjut ke tahap semantic analysis pada milestone berikutnya.

Tujuan berikutnya adalah merumuskan dan mengimplementasikan grammar bahasa Arion secara terstruktur di dalam program C/C++. Grammar tersebut harus

mencakup seluruh konstruksi yang ditentukan dalam spesifikasi Milestone 2, sehingga parser memiliki cakupan yang tepat terhadap fitur bahasa Arion. Dengan implementasi yang terstruktur, grammar akan mudah ditelusuri dan dijadikan acuan saat melakukan pengujian.

Laporan ini juga bertujuan menghasilkan parse tree yang merepresentasikan struktur hierarkis program Arion secara lengkap. Parse tree harus dapat dicetak ke terminal dan disimpan ke file berformat .txt sesuai spesifikasi, sehingga hasil parsing dapat diverifikasi secara manual maupun digunakan kembali pada tahap berikutnya.

Tujuan lain yang penting adalah menyediakan mekanisme error handling pada level parser. Parser harus mendeteksi syntax error, menandai posisi kesalahan, dan menampilkan pesan yang informatif agar proses debugging lebih efisien. Selain itu, tujuan akhir dari milestone ini adalah memastikan bahwa keluaran parse tree dapat digunakan sebagai masukan langsung untuk tahap semantic analysis yang akan datang.

1.4 Batasan Masalah

Pengerjaan Milestone 2 dibatasi oleh spesifikasi bahasa Arion yang ditetapkan pada dokumen resmi hingga Revisi 5 tertanggal 04-05-2026. Parser yang dibangun hanya mengakomodasi grammar dan aturan bahasa Arion pada milestone ini, sehingga tidak mencakup bahasa lain maupun fitur di luar spesifikasi. Batasan ini menjaga fokus implementasi agar sesuai dengan kebutuhan tugas besar.

Algoritma parsing yang digunakan terbatas pada Recursive Descent. Pendekatan ini dipilih karena kesederhanaan implementasi dan kesesuaiannya dengan grammar yang tersedia, sehingga algoritma lain seperti LR, LALR, atau Earley tidak digunakan. Konsekuensinya, grammar harus disesuaikan agar kompatibel dengan parsing top-down.

Keluaran utama parser adalah parse tree, bukan abstract syntax tree (AST). Parse tree memuat seluruh simbol terminal dan non-terminal sesuai grammar, sementara AST akan menjadi fokus pada milestone berikutnya. Selain itu, implementasi dilakukan dalam bahasa C/C++, melanjutkan struktur kode Milestone 1, agar integrasi antara lexer dan parser tetap konsisten.

Masukan parser dibatasi pada file daftar token hasil lexer Milestone 1, bukan langsung dari source code Arion. Meskipun secara pipeline lexer dan parser dijalankan berurutan, parser tetap menerima format token stream sesuai spesifikasi. Terakhir, parser pada milestone ini hanya melakukan pengecekan sintaksis dan belum mencakup analisis semantik seperti pengecekan tipe data atau scope variabel.

Bab 2

Dasar Teori

2.1 Teori Singkat

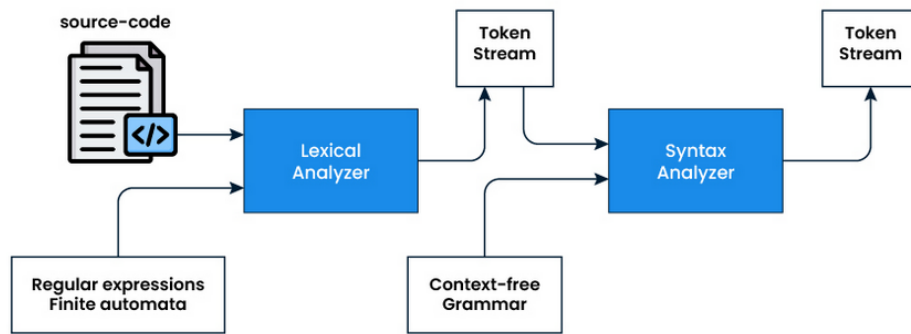
2.1.1 Parser dan Syntax Analysis

Parser atau syntax analyzer adalah komponen yang bertugas memeriksa struktur sintaks program berdasarkan aturan grammar bahasa. Setelah tahap lexical analysis selesai, parser menerima urutan token dan memverifikasi bahwa token-token tersebut dapat dibentuk menjadi konstruksi sintaks yang sah. Dengan kata lain, parser memeriksa apakah program mengikuti aturan bahasa, bukan lagi sekadar pola karakter.

Dalam pipeline kompilasi yang umum, posisi parser berada setelah lexer. Alur ini dapat diringkas sebagai berikut: source code diproses oleh lexer menjadi daftar token, kemudian parser mengolah token tersebut menjadi parse tree, yang selanjutnya dipakai oleh semantic analysis dan code generation. Parser tidak menerima source code mentah, melainkan daftar token yang sudah diklasifikasikan oleh lexer. Hubungan ini membuat output lexer pada Milestone 1 menjadi input utama parser pada Milestone 2.

Peran parser bersifat krusial karena ia menegakkan aturan grammar dan membangun representasi hierarkis dari program. Parser memeriksa urutan token apakah sesuai dengan produksi grammar, mendeteksi kesalahan sintaks, serta menyusun struktur parse tree yang menunjukkan bagaimana bagian-bagian program saling terhubung. Representasi ini menjadi dasar untuk analisis semantik dan tahapan kompilasi berikutnya.

Secara umum, parser dapat dibedakan menjadi pendekatan top-down dan bottom-up. Top-down memulai analisis dari simbol awal grammar lalu menurunkannya menjadi token yang cocok, sedangkan bottom-up membangun struktur dari token menuju simbol awal. Recursive Descent yang digunakan pada Arion Compiler adalah bentuk top-down parsing. Ketika terjadi syntax error, parser harus mampu memberikan pesan kesalahan yang informatif, misalnya menjelaskan token yang tidak sesuai dan posisi baris serta kolomnya agar proses debugging menjadi lebih efektif.

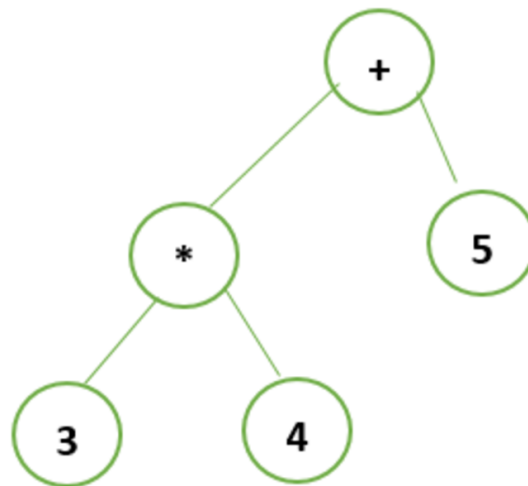


Gambar 2.1: Diagram pipeline kompilasi untuk syntax analyzer.

2.1.2 Parse Tree

Parse tree adalah representasi hierarkis dari struktur sintaks program berdasarkan grammar. Setiap simpul pada parse tree merepresentasikan penerapan aturan produksi sehingga hubungan antar bagian program dapat terlihat secara jelas. Parse tree menampilkan bagaimana sebuah program dibangun dari simbol awal hingga menghasilkan urutan token yang sesuai dengan grammar.

Dalam teori CFG, komponen parse tree dapat dijelaskan sebagai berikut. Node interior diberi label non-terminal, node daun diberi label terminal atau epsilon, dan root diberi label start symbol grammar. Setiap node interior A dengan anak $X_1, X_2, \dots, X_k$ menggambarkan produksi $A \rightarrow X_1X_2\dots X_k$. Dengan struktur ini, parse tree memberikan gambaran eksplisit dari proses derivasi yang menghasilkan program.

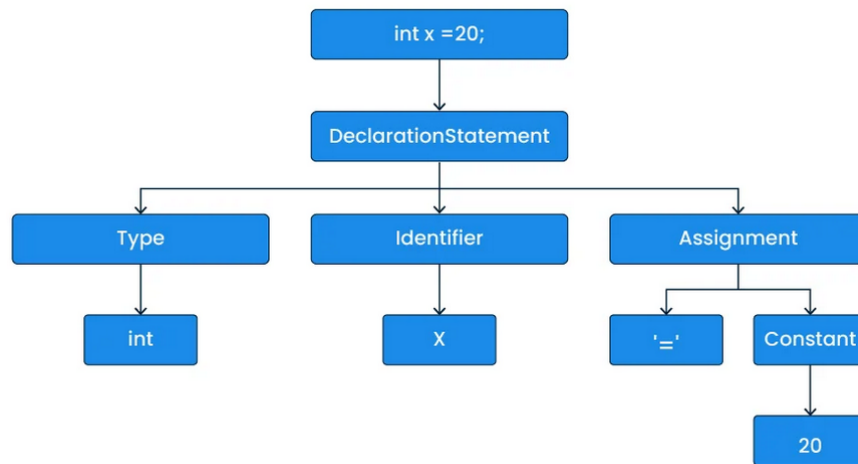


Gambar 2.2: Contoh syntax tree sederhana.

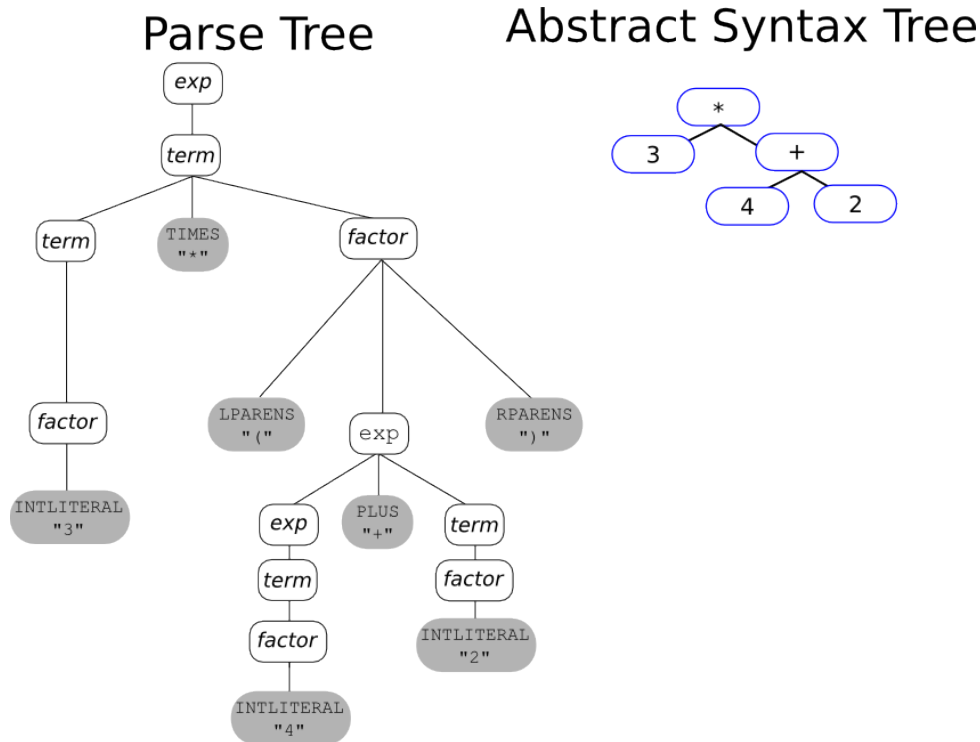
Konsep yield pada parse tree merujuk pada string terminal yang diperoleh de-

ngan membaca daun dari kiri ke kanan. Yield ini harus sama dengan urutan token input agar parse tree dianggap valid. Parse tree juga merupakan representasi alternatif dari derivasi leftmost atau rightmost, karena setiap derivasi tersebut dapat dipetakan ke satu parse tree yang setara secara struktural.

Parse tree berbeda dengan Abstract Syntax Tree (AST). Parse tree memuat semua simbol, termasuk tanda baca dan token terminal yang berfungsi sebagai struktur sintaks. AST mengabstraksi detail sintaks tersebut dan hanya menyimpan informasi semantis penting. Pada Milestone 2 ini, keluaran parser adalah parse tree lengkap, bukan AST. Parse tree penting karena menjadi input untuk semantic analysis dan menyediakan struktur program yang terverifikasi secara sintaks.



Gambar 2.3: Contoh parse tree untuk ekspresi aritmatika sederhana dengan label node terminal dan non-terminal.



Gambar 2.4: Perbandingan parse tree dan AST untuk ekspresi yang sama; parse tree lebih lengkap dengan semua terminal.

2.1.3 Recursive Descent Parser

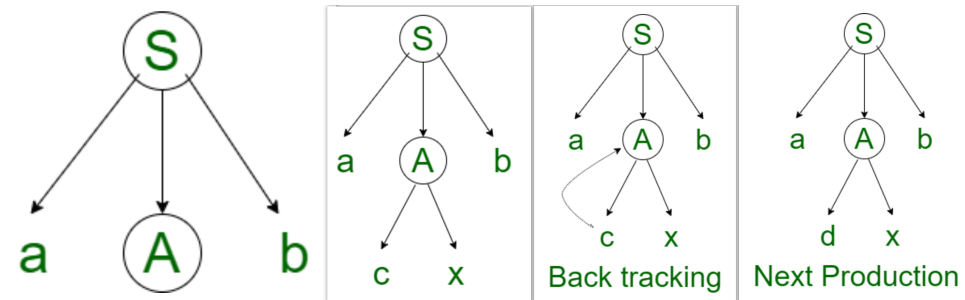
Recursive Descent Parser adalah metode parsing top-down yang mengimplementasikan satu fungsi rekursif untuk setiap non-terminal grammar. Ide utamanya adalah merepresentasikan aturan grammar sebagai prosedur yang secara langsung memeriksa token input dan membangun struktur parse tree. Pendekatan ini banyak digunakan karena sederhana dan mudah dipetakan ke kode program.

Pada top-down parsing, proses dimulai dari start symbol, lalu diperluas ke bawah mengikuti aturan produksi sampai mencapai terminal yang cocok dengan token input. Setiap fungsi non-terminal akan memeriksa token saat ini, memilih produksi yang relevan, lalu memanggil fungsi lain sesuai urutan simbol dalam produksi. Ketika sebuah terminal diharapkan, parser mencocokkan token input; jika cocok, token dikonsumsi dan parser maju ke token berikutnya.

Cara kerja recursive descent dapat dijelaskan langkah demi langkah. Setiap non-terminal memiliki fungsi `parseX()` yang membaca token, memanggil `parseY()` untuk non-terminal lain, dan mengonsumsi token ketika bertemu terminal yang diharapkan. Jika token tidak cocok, parser melempar syntax error. Fungsi-fungsi ini dipanggil secara rekursif sehingga struktur pemanggilan mengikuti struktur grammar. Contoh sederhana pada grammar ekspresi aritmatika adalah `parseExpression()` memanggil `parseTerm()`, dan `parseTerm()` memanggil `parseFactor()`.

Metode ini memiliki kelebihan karena mudah diimplementasikan dan struktur kodenya mencerminkan grammar. Namun, ia tidak dapat menangani grammar yang left-recursive secara langsung sehingga grammar harus ditransformasikan terlebih

dahulu. Selain itu, recursive descent umumnya bersifat deterministik dan mengandalkan lookahead terbatas, seperti LL(1). Dalam Arion Compiler, grammar bahasa di-hardcode ke dalam fungsi `parseX()` seperti `parseProgram()`, `parseDeclarationPart()`, `parseCompoundStatement()`, `parseStatement()`, dan `parseExpression()`, sehingga struktur parser menjadi jelas dan teruji.



Gambar 2.5: Ilustrasi proses recursive descent sebagai pohon pemanggilan fungsi selama parsing.

Bab 3

Perancangan dan Implementasi

3.1 Gambaran Umum Program

Implementasi Milestone 2 melanjutkan struktur program dari Milestone 1 dengan menambahkan komponen parser pada pipeline kompilasi. Alur utama program berada pada berkas `src/main.cpp`, yang bertugas membaca masukan, menjalankan lexer bila masukan berupa source code, atau langsung memuat token bila masukan sudah berbentuk token stream. Setelah token tersedia, parser membangun parse tree dan hasilnya dicetak ke terminal serta dapat disimpan ke file keluaran. Desain ini menjaga agar pipeline tetap fleksibel untuk kebutuhan pengujian: program dapat diuji langsung dari token stream hasil Milestone 1 maupun dari source code Arion.

Modul lexer tetap digunakan untuk mengubah source code menjadi daftar token, sedangkan modul `token_stream_reader` berfungsi mengonversi file token dump menjadi objek token yang bisa diproses parser. Modul parser mengandung implementasi Recursive Descent untuk seluruh non-terminal pada grammar Arion, dan modul parse tree menyimpan struktur node beserta mekanisme pencetakan pohon. Struktur ini mencerminkan spesifikasi pada `spek2.md` dan menjaga pemisahan tanggung jawab antar komponen, sehingga parser dapat fokus pada verifikasi sintaks dan penyusunan parse tree.

3.2 Algoritma Parser

Algoritma parser yang digunakan adalah Recursive Descent dengan satu fungsi untuk setiap non-terminal grammar. Parser membaca token secara berurutan dengan strategi top-down, dimulai dari simbol awal `<program>`. Pada setiap fungsi, parser memeriksa token saat ini menggunakan lookahead sederhana, memilih produksi yang relevan, lalu mengonsumsi token jika cocok. Jika token tidak sesuai, parser menghentikan proses dan mengeluarkan pesan kesalahan yang memuat lokasi baris dan kolom.

Logika pencocokan token dibangun melalui tiga operasi utama: `check` untuk melakukan inspeksi token tanpa menggeser posisi, `consume` untuk memvalidasi dan mengonsumsi token yang diharapkan, serta `advance` untuk berpindah token ketika

tipe sudah cocok. Ketika terjadi mismatch, parser memanggil `throwSyntaxError` agar pesan error konsisten di seluruh aturan. Pola ini membuat alur parsing mudah diikuti, karena setiap fungsi hanya berfokus pada aturan produksinya sendiri.

3.3 Struktur Direktori Program

Struktur direktori program pada Milestone 2 mengikuti pola yang sama dengan Milestone 1, dengan penambahan modul parser. Penjelasan ringkas struktur direktori utama ditunjukkan berikut.

- **src/**. Direktori utama yang berisi seluruh source code program dalam bahasa C++. Direktori ini terbagi menjadi beberapa modul.
 - **main.cpp**. Entry point program. Bertanggung jawab membaca input, menentukan apakah input berupa token stream atau source code, menjalankan lexer bila diperlukan, memanggil parser, serta mencetak dan menyimpan parse tree.
 - **lexer/**. Subdirektori lexer dan token, berisi:
 - * **token.h** dan **token.cpp**. Definisi `TokenType`, struktur `Token`, serta fungsi utilitas format token.
 - * **lexer.h** dan **lexer.cpp**. Implementasi DFA lexer untuk menghasilkan token dari source code.
 - * **token_stream_reader.h** dan **token_stream_reader.cpp**. Parser token stream yang mengubah output Milestone 1 menjadi objek token.
 - **parser/**. Subdirektori parser, berisi:
 - * **parser.h** dan **parser.cpp**. Implementasi Recursive Descent Parser untuk seluruh non-terminal grammar Arion.
 - * **parse_tree_node.h** dan **parse_tree_node.cpp**. Definisi node parse tree dan fungsi pencetakan pohon.
 - * **parse_tree_utils.h** dan **parse_tree_utils.cpp**. Kumpulan utilitas traversal dan pencarian node parse tree.
- **doc/**. Direktori dokumentasi proyek yang berisi laporan Milestone 2 dalam format LaTeX.
- **test/**. Direktori test case untuk pengujian program. Di dalamnya terdapat subdirektori **milestone-2/** yang menyimpan input dan output pengujian parser.
- **Makefile**. Berisi rule kompilasi program.
- **README.md**. Dokumentasi ringkas program, cara build/run, serta pembagian tugas.

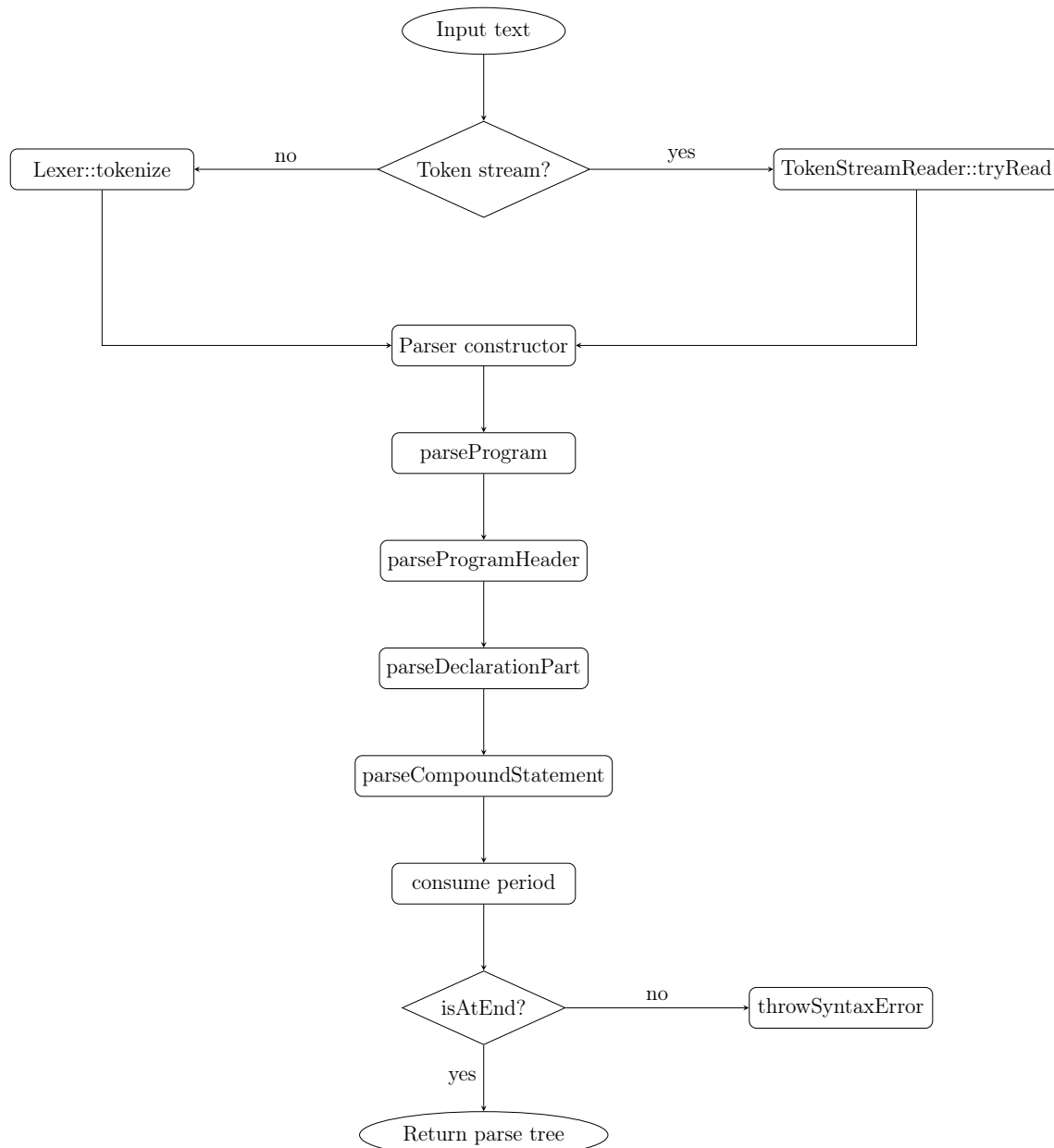
3.4 Implementasi Recursive Descent Parser

Implementasi Recursive Descent diwujudkan dalam `parser.cpp` dan `parser.h`. Modul ini membangun parse tree dengan memetakan setiap non-terminal grammar ke fungsi rekursif, serta menegakkan aturan token yang diharapkan melalui mekanisme `check`, `consume`, dan `throwSyntaxError`.

3.4.1 Alur Utama Parsing

Parser dibentuk dari token hasil lexer atau token stream. Konstruktor `Parser` menyaring token `COMMENT` dan menolak token `ERROR/UNKNOWN` agar masalah leksikal tidak merusak tahap parsing. Entry point parsing adalah `parseProgram()`, yang menyusun urutan `program-header`, `declaration-part`, `compound-statement`, dan token `period`. Setelah itu parser memastikan tidak ada token tersisa. Skema ini mengikuti aturan produksi utama pada grammar Arion, menjaga determinisme, dan memastikan program selalu ditutup dengan `period`.

Flowchart Diagram berikut merangkum alur kerja parser dari input hingga parse tree terbentuk.



Gambar 3.1: Flowchart alur utama parser

Pseudocode Berikut ringkasan algoritma dari `parseProgram()` berdasarkan implementasi aktual.

Pseudocode `parseProgram`

```
function parseProgram(tokens):  
    parser = Parser(tokens)  
    node = ProgramNode()  
    node.add(parseProgramHeader())  
    node.add(parseDeclarationPart())  
    node.add(parseCompoundStatement())
```

```
node.add(terminal(consume(PERIOD)))  
if not isAtEnd():  
    throwSyntaxError("end of input")  
return node
```

3.4.2 Pemilihan Produksi Dengan Lookahead

Pemilihan produksi dilakukan dengan lookahead satu token melalui `check` dan variasi `check(..., offset)`. Strategi ini dipakai ketika satu token awal bisa mengarah ke beberapa produksi. Contohnya, pada `parseStatement` parser membedakan assignment dan procedure/function call hanya dari token `ident` yang diikuti `becomes`, `lbrack`, atau `period`. Pola yang sama dipakai pada `parseFactor` untuk membedakan pemanggilan prosedur/fungsi, akses variabel berkomponen, dan `ident` biasa. Dengan lookahead satu token, parser tidak memerlukan backtracking sehingga tetap sederhana dan efisien.

Pseudocode lookahead

```
function check(type, offset = 0):  
    token = peek(offset)  
    return token != null and token.type == type  
  
function parseStatement():  
    if check(IDENT):  
        if check(BECOMES, 1) or check(LBRACK, 1) or check(PERIOD, 1):  
            return parseAssignmentStatement()  
        else:  
            return parseProcedureFunctionCall()  
    ...
```

3.4.3 Struktur Fungsi Per Non-Terminal

Non-terminal deklarasi seperti `const-declaration`, `type-declaration`, dan `var-declaration` diimplementasikan sebagai loop agar dapat menerima jumlah deklarasi yang bervariasi. Non-terminal statement seperti `if`, `case`, `while`, `repeat`, dan `for` dipecah ke fungsi khusus agar struktur grammar tercermin langsung di dalam kode. Dengan pola ini, urutan pemanggilan fungsi parser menyerupai proses derivasi grammar secara eksplisit.

Fungsi dispatcher seperti `parseType()` dan `parseStatement()` memilih cabang produksi berdasarkan token saat ini. Non-terminal yang opsional (misalnya `statement kosong`) ditangani dengan pengecekan follow set sederhana, sehingga parser tetap konsisten dengan grammar tanpa memaksakan token yang tidak ada.

Pseudocode struktur non-terminal

```
function parseDeclarationPart():
    node = DeclarationPartNode()
    while check(CONSTSY):
        node.add(parseConstDeclaration())
    while check(TYPE SY):
        node.add(parseTypeDeclaration())
    while check(VARSY):
        node.add(parseVarDeclaration())
    while check(PROCEDURESY) or check(FUNCTIONSY):
        node.add(parseSubprogramDeclaration())
    return node

function parseStatement():
    node = StatementNode()
    if check(IDENT):
        if check(BECOMES, 1) or check(LBRACK, 1) or check(PERIOD, 1):
            node.add(parseAssignmentStatement())
        else:
            node.add(parseProcedureFunctionCall())
    return node
    if check(BEGINSY): return node.add(parseCompoundStatement())
    if check(IFS Y): return node.add(parseIfStatement())
    if check(CASESY): return node.add(parseCaseStatement())
    if check(WHILESY): return node.add(parseWhileStatement())
    if check(REPEATSY): return node.add(parseRepeatStatement())
    if check(FORSY): return node.add(parseForStatement())
    if isStatementFollow(): return node.add(EmptyStatementNode())
    throwSyntaxError("statement")
```

3.4.4 Hierarki Ekspresi dan Prioritas Operator

Ekspresi dipecah menjadi empat level: *expression*, *simple-expression*, *term*, dan *factor*. Pemisahan ini memastikan prioritas operator diproses dengan benar. Operator relasional ditangani pada *expression*, operator aditif pada *simple-expression*, dan operator multiplikatif pada *term*. *factor* menangani literal, variabel, pemanggilan prosedur/fungsi, serta ekspresi di dalam tanda kurung.

Struktur ini menjaga agar ekspresi seperti $a + b * c$ diproses sesuai precedence, karena `parseTerm()` selalu menyelesaikan operasi multiplikatif sebelum `parseSimpleExpression()` menggabungkannya dengan operator aditif.

Pseudocode hierarki ekspresi

```
function parseExpression():
    node = ExpressionNode()
```

```
node.add(parseSimpleExpression())
if next is relop:
    node.add(parseRelationalOperator())
    node.add(parseSimpleExpression())
return node

function parseSimpleExpression():
    node = SimpleExpressionNode()
    if next is PLUS or MINUS:
        node.add(terminal(advance()))
    node.add(parseTerm())
    while next is PLUS or MINUS or ORSY:
        node.add(parseAdditiveOperator())
        node.add(parseTerm())
    return node

function parseTerm():
    node = TermNode()
    node.add(parseFactor())
    while next is TIMES or RDIV or IDIV or IMOD or ANDSY:
        node.add(parseMultiplicativeOperator())
        node.add(parseFactor())
    return node

function parseFactor():
    if next is IDENT:
        if lookahead is LPARENT:
            return parseProcedureFunctionCall()
        if lookahead is LBRACK or PERIOD:
            return parseVariable()
        return terminal(advance())
    if next is literal:
        return terminal(advance())
    if next is LPARENT:
        advance(); node = parseExpression(); consume(RPARENT)
    return node
    if next is NOTSY:
        advance(); return parseFactor()
    throwSyntaxError("factor")
```

3.4.5 Variabel Dengan Komponen Berantai

Parser mendukung akses array dan record melalui `parseVariableWithBase` yang membungkus variabel dasar ke dalam `component-variable` secara rekursif. Variabel dasar selalu diawali `ident`. Setiap suffix berupa index `[ ... ]` atau akses

field `.ident` menghasilkan node baru yang menampung base variable sebelumnya, sehingga konstruk seperti `arr[1].field` diuraikan sebagai rantai node bertingkat. Untuk Milestone 2, isi `index-list` dibatasi pada `intcon`, `charcon`, atau `ident`.

Pseudocode variable berantai

```
function parseVariable():
    base = VariableNode()
    base.add(terminal(consume(IDENT)))
    return parseVariableWithBase(base)

function parseVariableWithBase(base):
    if not (next is LBRACK or next is PERIOD):
        return base
    wrapped = VariableNode()
    wrapped.add(parseComponentVariable(base))
    return parseVariableWithBase(wrapped)

function parseComponentVariable(base):
    node = ComponentVariableNode()
    node.add(base)
    if next is LBRACK:
        node.add(terminal(advance()))
        node.add(parseIndexList())
        node.add(terminal(consume(RBRACK)))
        return node
    node.add(terminal(consume(PERIOD)))
    node.add(terminal(consume(IDENT)))
    return node

function parseIndexList():
    node = IndexListNode()
    require next in {INTCON, CHARCON, IDENT}
    node.add(terminal(advance()))
    while next is COMMA:
        node.add(terminal(advance()))
        require next in {INTCON, CHARCON, IDENT}
        node.add(terminal(advance()))
    return node
```

3.4.6 Penanganan Error dan Validasi Token

Penanganan error dipusatkan pada `throwSyntaxError`, yang menghasilkan pesan berformat konsisten dengan informasi baris dan kolom. Parser juga memfilter token komentar serta menolak token `ERROR` dan `UNKNOWN` sejak konstruksi parser, sehingga masalah leksikal tidak merusak tahap parsing. Dengan cara ini, parser berhenti secara deterministik ketika menemukan pelanggaran grammar dan memberikan

diagnosis yang jelas.

3.5 Implementasi Kode

Bagian ini menjelaskan isi setiap file dengan memisahkan penjelasan per snippet penting agar struktur implementasi mudah ditelusuri.

3.5.1 main.cpp

Alur Utama Eksekusi

Snippet berikut menunjukkan alur utama: validasi argumen, pembacaan input, pemilihan jalur token stream atau lexer, pemanggilan parser, dan pencetakan output.

```
1 if (argc < 2) {
2     std::cerr << "Usage: " << argv[0] << " <input.txt>
      [output.txt]" << std::endl;
3     return 1;
4 }
5
6 Lexer lexer(source);
7     if (!TokenStreamReader::tryRead(source, tokens,
      tokenStreamError)) {
8         std::cerr << "Error: " << tokenStreamError <<
          std::endl;
9         return 1;
10    }
11 } else {
12     tokens = lexer.tokenize();
13     if (lexer.hasErrors()) {
14         return 1;
15    }
16 }
17
18 Parser parser(tokens);
19 std::unique_ptr<ProgramNode> parseTree =
      parser.parseProgram();
20 parseTree->print(outputBuffer);
```

Listing 3.1: Alur utama program pada main.cpp

3.5.2 lexer/token.h

Enum TokenType

Enum ini mendefinisikan seluruh jenis token yang dipakai lexer dan parser.

```
1 enum class TokenType {
2     INTCON, REALCON, CHARCON, STRING,
```

```
3     NOTSY, ANDSY, ORSY,  
4     PLUS, MINUS, TIMES, IDIV, RDIV, IMOD,  
5     EQL, NEQ, GTR, GEQ, LSS, LEQ,  
6     LPARENT, RPARENT, LBRACK, RBRACK, COMMA,  
7     SEMICOLON, PERIOD, COLON, BECOMES,  
8     CONSTSY, TYPESY, VARSY, FUNCTIONSY, PROCEDURESY,  
9     ARRAYSY, RECORDSY, PROGRAMSY,  
10    IDENT, BEGINSY, IFSY, CASESY, REPEATSY,  
11    WHILESY, FORSY, ENDSY, ELSESY, UNTILSY,  
12    OFSY, DOSY, TOSY, DOWNTOSY, THENSY,  
13    COMMENT, UNKNOWN, ERROR  
14 };
```

Listing 3.2: Enum TokenType pada token.h

Struct Token

Struct ini menyimpan informasi token beserta lokasi baris dan kolom.

```
1 struct Token {  
2     TokenType type;  
3     std::string value;  
4     int line;  
5     int column;  
6 };
```

Listing 3.3: Struct Token pada token.h

3.5.3 lexer/token.cpp

Format Output Token

Fungsi berikut memastikan token dicetak sesuai format spesifikasi.

```
1 std::string formatToken(const Token& token) {  
2     if (tokenHasValue(token.type)) {  
3         return tokenTypeToString(token.type) +  
4             " (" + token.value + ")";  
5     }  
6     return tokenTypeToString(token.type);  
7 }
```

Listing 3.4: Implementasi formatToken pada token.cpp

3.5.4 lexer/token_stream_reader.h

Antarmuka Pembaca Token Stream

Header ini mengekspos fungsi utilitas untuk mendeteksi dan membaca token stream.

```
1 namespace TokenStreamReader {  
2     bool looksLikeTokenStream(const std::string &text);  
3     bool tryRead(const std::string &text,  
4                 std::vector<Token> &tokens,  
5                 std::string &errorMessage);  
6 }
```

Listing 3.5: Deklarasi TokenStreamReader

3.5.5 lexer/token_stream_reader.cpp

Pemetaan String ke TokenType

Token stream menggunakan label string yang dipetakan ke enum TokenType.

```
1 const std::unordered_map<std::string, TokenType>  
   &tokenTypeMap()  
2 {  
3     static const std::unordered_map<std::string, TokenType>  
       mapping = {  
4         {"intcon", TokenType::INTCON},  
5         {"realcon", TokenType::REALCON},  
6         {"charcon", TokenType::CHARCON},  
7         {"string", TokenType::STRING},  
8         {"programsy", TokenType::PROGRAMSY},  
9         {"ident", TokenType::IDENT},  
10        {"semicolon", TokenType::SEMICOLON},  
11        {"period", TokenType::PERIOD},  
12        {"error", TokenType::ERROR},  
13    };  
14    return mapping;  
15 }
```

Listing 3.6: Pemetaan token pada token_stream_reader.cpp

Parsing Token Stream

Fungsi `parseSequence` melakukan parsing token stream secara rekursif hingga akhir input.

```
1 bool parseSequence(std::size_t position, std::vector<Token>  
   &tokens) const  
2 {  
3     position = skipSeparators(position);  
4     if (position >= text.size()) {  
5         tokens.clear();  
6         return true;  
7     }  
8     return parseSequence(nextPosition, tokens);
```

9 }

Listing 3.7: Bagian parseSequence pada token_stream_reader.cpp

3.5.6 parser/parser.h

Kelas ParserError

Kelas ini menyimpan pesan error dan lokasi sumber error.

```
1 class ParserError : public std::runtime_error {
2 public:
3     ParserError(std::string message, int line, int column);
4     int line() const;
5     int column() const;
6 private:
7     int errorLine;
8     int errorColumn;
9 };
```

Listing 3.8: ParserError pada parser.h

Kelas Parser

Kelas ini mendeklarasikan seluruh fungsi non-terminal dan utilitas parsing.

```
1 class Parser {
2 public:
3     explicit Parser(const std::vector<Token> &tokens);
4     std::unique_ptr<ProgramNode> parseProgram();
5     std::unique_ptr<ProgramHeaderNode> parseProgramHeader();
6 private:
7     std::vector<Token> parserTokens;
8     std::size_t currentIndex;
9     bool check(TokenType type) const;
10    Token consume(TokenType type, const std::string
        &expectedName);
11    [[noreturn]] void throwSyntaxError(const std::string
        &expectedName) const;
12    // ... deklarasi fungsi non-terminal lainnya ...
13 };
```

Listing 3.9: Deklarasi inti Parser pada parser.h

3.5.7 parser/parser.cpp

Entry Point Parsing

Fungsi berikut memulai parsing dan memastikan program diakhiri token period.

```
1 std::unique_ptr<ProgramNode> Parser::parseProgram()  
2 {  
3     auto programNode = std::make_unique<ProgramNode>();  
4     programNode->addChild(parseProgramHeader());  
5     programNode->addChild(parseDeclarationPart());  
6     programNode->addChild(parseCompoundStatement());  
7     programNode->addChild(makeTerminalNode(consume(  
8         TokenType::PERIOD, "period")));  
9  
10    if (!isAtEnd()) {  
11        throwSyntaxError("end of input");  
12    }  
13    return programNode;  
14 }
```

Listing 3.10: parseProgram pada parser.cpp

Pelaporan Syntax Error

Penanganan error terpusat pada `throwSyntaxError` agar format pesan konsisten.

```
1 void Parser::throwSyntaxError(const std::string  
2     &expectedName) const  
3 {  
4     std::ostringstream message;  
5     if (isAtEnd()) {  
6         int line = parserTokens.empty() ? 1 :  
7             parserTokens.back().line;  
8         int column = parserTokens.empty() ? 1 :  
9             parserTokens.back().column;  
10        message << "Syntax error at line " << line  
11            << ", column " << column  
12            << ": unexpected end of input, expected " <<  
13            expectedName;  
14        throw ParserError(message.str(), line, column);  
15    }  
16  
17    const Token &token = peek();  
18    message << "Syntax error at line " << token.line  
19        << ", column " << token.column  
20        << ": unexpected token " <<  
21        tokenTypeToString(token.type)  
22        << ", expected " << expectedName;  
23    throw ParserError(message.str(), token.line,  
24        token.column);  
25 }
```

Listing 3.11: throwSyntaxError pada parser.cpp

Pemilihan Statement

Parser membedakan assignment dan procedure/function call berdasarkan lookahead.

```
1 if (check(TokenType::IDENT)) {  
2     if (check(TokenType::BECOMES, 1) ||  
3         check(TokenType::LBRACK, 1) ||  
4         check(TokenType::PERIOD, 1)) {  
5         statementNode->addChild(parseAssignmentStatement());  
6     } else {  
7         statementNode->addChild(parseProcedureFunctionCall());  
8     }  
9     return statementNode;  
10 }
```

Listing 3.12: Bagian parseStatement pada parser.cpp

3.5.8 parser/parse_tree_node.h

Struktur Dasar Parse Tree

Kelas berikut adalah fondasi semua node pada parse tree.

```
1 class ParseTreeNode {  
2 public:  
3     explicit ParseTreeNode(std::string name);  
4     const std::string &name() const;  
5     void addChild(std::unique_ptr<ParseTreeNode> child);  
6     void print(std::ostream &out, const std::string &prefix =  
7         "", bool isLast = true) const;  
8 protected:  
9     virtual std::string renderLabel() const;  
10 private:  
11     std::string nodeName;  
12     ParseTreeNode *parentNode;  
13     std::vector<std::unique_ptr<ParseTreeNode>> childNodes;  
14 };
```

Listing 3.13: ParseTreeNode pada parse_tree_node.h

Terminal Node

Node terminal menyimpan token dan mencetak label token saat output.

```
1 class ParseTreeTerminalNode : public ParseTreeNode {  
2 public:  
3     explicit ParseTreeTerminalNode(const Token &token);  
4     const Token &token() const;  
5 protected:  
6     std::string renderLabel() const override;
```

```
7 private:
8     Token terminalToken;
9 };
```

Listing 3.14: ParseTreeTerminalNode pada parse_tree_node.h

3.5.9 parser/parse_tree_node.cpp

Pencetakan Tree Dengan Indentasi

Fungsi print membangun struktur pohon dengan simbol garis.

```
1 void ParseTreeNode::print(std::ostream &out, const
2     std::string &prefix, bool isLast) const
3 {
4     out << prefix;
5     if (parentNode != nullptr) {
6         out << (isLast ? "`-- " : "|-- ");
7     }
8     out << renderLabel() << '\n';
9
10    const std::string childPrefix = prefix + (parentNode ==
11        nullptr ? "" : (isLast ? " " : "| "));
12    for (std::size_t index = 0; index < childNodes.size();
13        ++index) {
14        const bool childIsLast = index + 1 ==
15            childNodes.size();
16        childNodes[index]->print(out, childPrefix,
17            childIsLast);
18    }
19 }
```

Listing 3.15: ParseTreeNode::print pada parse_tree_node.cpp

3.5.10 parser/parse_tree_utils.h

Deklarasi Utilitas Traversal

Header ini menyediakan fungsi bantu untuk inspeksi pohon.

```
1 namespace ParseTreeUtils {
2     bool isRoot(const ParseTreeNode *node);
3     bool isLeaf(const ParseTreeNode *node);
4     std::vector<ParseTreeNode *>
5         collectAllNodes(ParseTreeNode *node);
6     std::vector<ParseTreeNode *>
7         collectTerminalNodes(ParseTreeNode *node);
8     std::vector<ParseTreeNode *>
9         collectNodesByName(ParseTreeNode *node, const
10             std::string &name);
11 }
```

7 }
}

Listing 3.16: Deklarasi utilitas parse tree

3.5.11 parser/parse_tree_utils.cpp

Contoh Traversal Preorder

Contoh berikut memperlihatkan traversal preorder untuk mengumpulkan seluruh node.

```
1 std::vector<ParseTreeNode *> collectAllNodes(ParseTreeNode
2     *node)
3 {
4     std::vector<ParseTreeNode *> result;
5     if (!node)
6         return result;
7
8     result.push_back(node);
9     for (auto &child : node->children()) {
10         auto childNodes = collectAllNodes(child.get());
11         result.insert(result.end(), childNodes.begin(),
12             childNodes.end());
13     }
14     return result;
15 }
```

Listing 3.17: collectAllNodes pada parse_tree_utils.cpp

Bab 4

Pengujian

4.1 Kasus Dasar

Tabel berikut merangkum file pengujian kasus dasar. Isi lengkap setiap file `input` dan `output` ditampilkan pada bagian detail setelah tabel.

Tabel 4.1: Ringkasan Pengujian pada Kasus Dasar

No	Test Case	Hasil
1	test/milestone-2/input-1.txt	test/milestone-2/output-1.txt
2	test/milestone-2/input-2.txt	test/milestone-2/output-2.txt
3	test/milestone-2/input-3.txt	test/milestone-2/output-3.txt

Detail Kasus Dasar

Kasus 1

Input

```
program Hello;  
  
var  
  a, b: integer;  
  
begin  
  a := 5;  
  b := a + 10;  
  writeln('Result = ', b);  
end.
```

Hasil

```
<program>  
|-- <program-header>  
|   |-- programsy  
|   |-- ident(Hello)  
|   '-- semicolon  
|-- <declaration-part>  
|   '-- <var-declaration>  
|       |-- varsy  
|       |-- <identifier-list>  
|           |-- ident(a)  
|           |-- comma
```

```
|      |-- ident(b)
|      |-- colon
|      |-- <type>
|      |-- ident(integer)
|      |-- semicolon
|-- <compound-statement>
|  |-- beginsy
|  |-- <statement-list>
|  |-- <statement>
|  |  |-- <assignment-statement>
|  |  |  |-- <variable>
|  |  |  |  |-- ident(a)
|  |  |  |-- becomes
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- intcon(5)
|  |  |-- semicolon
|  |-- <statement>
|  |  |-- <assignment-statement>
|  |  |  |-- <variable>
|  |  |  |  |-- ident(b)
|  |  |  |-- becomes
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |  |-- ident(a)
|  |  |  |  |-- <additive-operator>
|  |  |  |  |  |-- plus
|  |  |  |  |-- <term>
|  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |-- intcon(10)
|  |  |-- semicolon
|  |-- <statement>
|  |  |-- <procedure/function-call>
|  |  |  |-- ident(writeln)
|  |  |  |-- lparent
|  |  |  |-- <parameter-list>
|  |  |  |  |-- <expression>
|  |  |  |  |  |-- <simple-expression>
|  |  |  |  |  |  |-- <term>
|  |  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |  |  |-- string('Result = ')
|  |  |  |  |-- comma
|  |  |  |-- <expression>
|  |  |  |  |-- <simple-expression>
|  |  |  |  |  |-- <term>
|  |  |  |  |  |  |-- <factor>
|  |  |  |  |  |  |  |-- ident(b)
|  |  |-- rparent
|  |-- semicolon
|-- endsy
|-- period
```

Kasus 2

Input

```
program Expressions;

var
  x, y: integer;

begin
  x := (10 + 5) - 3 * 2;
  y := x div 2;
```

```
if (x > 5) and (y <= 10) then
    writeln('OK');
if not (x == 0) then
    writeln('NonZero');
end.
```

Hasil

```
<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(Expressions)
|   '-- semicolon
|-- <declaration-part>
|   '-- <var-declaration>
|       |-- varsy
|       |-- <identifier-list>
|           |-- ident(x)
|           |-- comma
|           '-- ident(y)
|       |-- colon
|       |-- <type>
|           '-- ident(integer)
|       '-- semicolon
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|       |-- <statement>
|           '-- <assignment-statement>
|               |-- <variable>
|               |   '-- ident(x)
|               |-- becomes
|               '-- <expression>
|                   '-- <simple-expression>
|                       |-- <term>
|                           '-- <factor>
|                               |-- lparent
|                               |-- <expression>
|                                   '-- <simple-expression>
|                                       |-- <term>
|                                           '-- <factor>
|                                               '-- intcon(10)
|                                       |-- <additive-operator>
|                                           '-- plus
|                                       '-- <term>
|                                           '-- <factor>
|                                               '-- intcon(5)
|                               '-- rparent
|                       |-- <additive-operator>
|                           '-- minus
|                   '-- <term>
|                       |-- <factor>
|                           '-- intcon(3)
|                       |-- <multiplicative-operator>
|                           '-- times
|                       '-- <factor>
|                           '-- intcon(2)
|               |-- semicolon
|       |-- <statement>
|           '-- <assignment-statement>
|               |-- <variable>
|               |   '-- ident(y)
|               |-- becomes
|               '-- <expression>
|                   '-- <simple-expression>
|                       '-- <term>
|                           |-- <factor>
|                           |   '-- ident(x)
|                           |-- <multiplicative-operator>
|                           |   '-- idiv
```

```
        '-- <factor>
        '-- intcon(2)
-- semicolon
-- <statement>
  '-- <if-statement>
    |-- ifsy
    |-- <expression>
      '-- <simple-expression>
        '-- <term>
          |-- <factor>
          |-- lparent
          |-- <expression>
            |-- <simple-expression>
              '-- <term>
                '-- <factor>
                  '-- ident(x)
            |-- <relational-operator>
              '-- gtr
          '-- <simple-expression>
            '-- <term>
              '-- <factor>
                '-- intcon(5)
          |-- rpparent
        '-- <multiplicative-operator>
          '-- andsy
      '-- <factor>
        |-- lparent
        |-- <expression>
          |-- <simple-expression>
            '-- <term>
              '-- <factor>
                '-- ident(y)
          |-- <relational-operator>
            '-- leq
        '-- <simple-expression>
          '-- <term>
            '-- <factor>
              '-- intcon(10)
        |-- rpparent
    |-- thensy
  '-- <statement>
    '-- <procedure/function-call>
      |-- ident(writeln)
      |-- lparent
      |-- <parameter-list>
      |-- <expression>
        '-- <simple-expression>
          '-- <term>
            '-- <factor>
              '-- string('OK')
      |-- rpparent
-- semicolon
-- <statement>
  '-- <if-statement>
    |-- ifsy
    |-- <expression>
      '-- <simple-expression>
        '-- <term>
          '-- <factor>
            |-- notsy
            '-- <factor>
              |-- lparent
              |-- <expression>
                |-- <simple-expression>
                  '-- <term>
                    '-- <factor>
                      '-- ident(x)
                |-- <relational-operator>
                  '-- eql
              |-- <simple-expression>
```

```
| | | | | '-- <term>  
| | | | | '-- <factor>  
| | | | | '-- intcon(0)  
| | | | |-- rparent  
|-- thensy  
|-- <statement>  
| | '-- <procedure/function-call>  
| | | |-- ident(writeln)  
| | | |-- lparent  
| | | |-- <parameter-list>  
| | | | '-- <expression>  
| | | | '-- <simple-expression>  
| | | | '-- <term>  
| | | | '-- <factor>  
| | | | '-- string('NonZero')  
| | |-- rparent  
|-- semicolon  
-- endsy  
-- period
```

Kasus 3

Input

```

program ControlFlow;

var
    i, x: integer;

begin
    while x < 10 do
        x := x + 1;
    for i := 1 to 3 do
        writeln(i);
    repeat
        x := x - 1
    until x == 0;
end.

```

Hasil

```

<program>
|-- <program-header>
|   |-- programsy
|   |-- ident(ControlFlow)
|   '-- semicolon
|-- <declaration-part>
|   '-- <var-declaration>
|       |-- varsy
|       |-- <identifier-list>
|           |-- ident(i)
|           |-- comma
|           '-- ident(x)
|       |-- colon
|       |-- <type>
|           '-- ident(integer)
|       '-- semicolon
|-- <compound-statement>
|   |-- beginsy
|   |-- <statement-list>
|       |-- <statement>
|           '-- <while-statement>
|               |-- whilesy
|               |-- <expression>
|                   |-- <simple-expression>
|                       '-- <term>
|                           '-- <factor>

```



```

|-- ident(x)
|-- <relational-operator>
|-- lss
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(10)
|-- dosy
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(x)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(x)
|-- <additive-operator>
|-- plus
|-- <term>
|-- <factor>
|-- intcon(1)
|-- semicolon
|-- <statement>
|-- <for-statement>
|-- forsy
|-- ident(i)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(1)
|-- tosy
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(3)
|-- dosy
|-- <statement>
|-- <procedure/function-call>
|-- ident(writeln)
|-- lparent
|-- <parameter-list>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(i)
|-- rparent
|-- semicolon
|-- <statement>
|-- <repeat-statement>
|-- repeatsy
|-- <statement-list>
|-- <statement>
|-- <assignment-statement>
|-- <variable>
|-- ident(x)
|-- becomes
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(x)
|-- <additive-operator>
|-- minus
|-- <term>
```

```

|-- <factor>
|-- intcon(1)
|-- untilsy
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- ident(x)
|-- <relational-operator>
|-- eql
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(0)
|-- semicolon
|-- endsy
|-- period

```

4.2 *Edge Cases*

Tabel berikut merangkum file pengujian edge cases. Isi lengkap setiap file `input` dan `output` ditampilkan pada bagian detail setelah tabel.

Tabel 4.2: Ringkasan Pengujian pada Kasus Anomali

No	Test Case	Hasil
1	test/milestone-2/input-4.txt	test/milestone-2/output-4.txt
2	test/milestone-2/input-5.txt	test/milestone-2/output-5.txt

Detail Edge Cases

Kasus 4

Input

```
program StructuredData;

type
  Student == record
    age: integer;
  end;

var
  data: Student;
  arr: array[1..10] of integer;

begin
  data.age := 1;
  arr[1] := data.age;
  case arr[1] of
    0: writeln('zero');
    1, 2: writeln('small');
  end;
end.
```

Hasil

```
<program>
|-- <program-header>
|   |-- programsy
```

```

|-- ident(StructuredData)
'-- semicolon
-- <declaration-part>
|-- <type-declaration>
|   |-- typesy
|   |-- ident(Student)
|   |-- eql
|   |-- <type>
|       |-- <record-type>
|           |-- recordsy
|           |-- <field-list>
|               |-- <field-part>
|                   |-- <identifier-list>
|                       |-- ident(age)
|                       |-- colon
|                       |-- <type>
|                           |-- ident(integer)
|                           |-- semicolon
|                       |-- endsy
|                   |-- semicolon
|       |-- <var-declaration>
|           |-- varsy
|           |-- <identifier-list>
|               |-- ident(data)
|               |-- colon
|               |-- <type>
|                   |-- ident(Student)
|                   |-- semicolon
|               |-- <identifier-list>
|                   |-- ident(arr)
|                   |-- colon
|                   |-- <type>
|                       |-- <array-type>
|                           |-- arraysy
|                           |-- lbrack
|                           |-- <range>
|                               |-- <constant>
|                                   |-- intcon(1)
|                                   |-- period
|                                   |-- period
|                                   |-- <constant>
|                                       |-- intcon(10)
|                           |-- rbrack
|                           |-- ofsy
|                           |-- <type>
|                               |-- ident(integer)
|                           |-- semicolon
|       |-- <compound-statement>
|           |-- beginsy
|           |-- <statement-list>
|               |-- <statement>
|                   |-- <assignment-statement>
|                       |-- <variable>
|                           |-- <component-variable>
|                               |-- <variable>
|                                   |-- ident(data)
|                                   |-- period
|                                   |-- ident(age)
|                       |-- becomes
|                       |-- <expression>
|                           |-- <simple-expression>
|                               |-- <term>
|                                   |-- <factor>
|                                       |-- intcon(1)
|                   |-- semicolon
|                   |-- <statement>
|                       |-- <assignment-statement>
|                           |-- <variable>
|                               |-- <component-variable>
|                                   |-- <variable>

```

```

|-- ident(arr)
|-- lbrack
|-- <index-list>
| '-- intcon(1)
|-- rbrack
|-- becomes
|-- <expression>
| '-- <simple-expression>
| | '-- <term>
| | | '-- <factor>
| | | | '-- <variable>
| | | | | '-- <component-variable>
| | | | | | '-- <variable>
| | | | | | '-- ident(data)
| | | | | | '-- period
| | | | | '-- ident(age)
|-- semicolon
|-- <statement>
| '-- <case-statement>
| | -- casesy
| | -- <expression>
| | | '-- <simple-expression>
| | | | '-- <term>
| | | | | '-- <factor>
| | | | | | '-- <variable>
| | | | | | | '-- <component-variable>
| | | | | | | | '-- <variable>
| | | | | | | | | '-- ident(arr)
| | | | | | | | -- lbrack
| | | | | | | | -- <index-list>
| | | | | | | | | '-- intcon(1)
| | | | | | | -- rbrack
| | -- ofsy
| | -- <case-block>
| | | -- <constant>
| | | | '-- intcon(0)
| | | -- colon
| | | -- <statement>
| | | | '-- <procedure/function-call>
| | | | | -- ident(writeln)
| | | | | -- lparent
| | | | | -- <parameter-list>
| | | | | | '-- <expression>
| | | | | | | '-- <simple-expression>
| | | | | | | | '-- <term>
| | | | | | | | | '-- <factor>
| | | | | | | | | | '-- string('zero')
| | | | | -- rparent
| | | -- semicolon
| | -- <case-block>
| | | -- <constant>
| | | | '-- intcon(1)
| | | -- comma
| | | -- <constant>
| | | | '-- intcon(2)
| | | -- colon
| | | -- <statement>
| | | | '-- <procedure/function-call>
| | | | | -- ident(writeln)
| | | | | -- lparent
| | | | | -- <parameter-list>
| | | | | | '-- <expression>
| | | | | | | '-- <simple-expression>
| | | | | | | | '-- <term>
| | | | | | | | | '-- <factor>
| | | | | | | | | | '-- string('small')
| | | | | -- rparent
| | | -- semicolon
|-- endsy
-- semicolon

```

```
|  '-- endsy  
'-- period
```

Kasus 5

Input

```
program Subprograms;  
  
procedure PrintTwice(x: integer);  
begin  
    writeln(x);  
    writeln(x);  
end;  
  
function IncOne(x: integer): integer;  
begin  
    IncOne := x + 1;  
end;  
  
begin  
    PrintTwice(2);  
    writeln(IncOne(5));  
end.
```

Hasil

```
<program>  
|-- <program-header>  
|   |-- programsy  
|   |-- ident(Subprograms)  
|   '-- semicolon  
|-- <declaration-part>  
|   |-- <subprogram-declaration>  
|   |   '-- <procedure-declaration>  
|   |   |   |-- proceduresy  
|   |   |   |-- ident(PrintTwice)  
|   |   |   |-- <formal-parameter-list>  
|   |   |   |   |-- lparent  
|   |   |   |   |-- <parameter-group>  
|   |   |   |   |   |-- <identifier-list>  
|   |   |   |   |   |   '-- ident(x)  
|   |   |   |   |   |-- colon  
|   |   |   |   |   '-- ident(integer)  
|   |   |   |   '-- rparent  
|   |   |-- semicolon  
|   |   |-- <block>  
|   |   |   |-- <declaration-part>  
|   |   |   '-- <compound-statement>  
|   |   |   |   |-- beginsy  
|   |   |   |   |-- <statement-list>  
|   |   |   |   |   |-- <statement>  
|   |   |   |   |   |   '-- <procedure/function-call>  
|   |   |   |   |   |   |   |-- ident(writeln)  
|   |   |   |   |   |   |   |-- lparent  
|   |   |   |   |   |   |   |-- <parameter-list>  
|   |   |   |   |   |   |   |   '-- <expression>  
|   |   |   |   |   |   |   |   |   '-- <simple-expression>  
|   |   |   |   |   |   |   |   |   |   '-- <term>  
|   |   |   |   |   |   |   |   |   |   |   '-- <factor>  
|   |   |   |   |   |   |   |   |   |   |   '-- ident(x)  
|   |   |   |   |   |   |   '-- rparent  
|   |   |   |   |-- semicolon  
|   |   |   |-- <statement>  
|   |   |   |   '-- <procedure/function-call>  
|   |   |   |   |   |-- ident(writeln)  
|   |   |   |   |-- lparent
```

```
| | | | | -- <parameter-list>
| | | | | ' -- <expression>
| | | | |   ' -- <simple-expression>
| | | | |     ' -- <term>
| | | | |       ' -- <factor>
| | | | |         ' -- ident(x)
| | | | |   ' -- rparent
| | | | | ' -- semicolon
| | | | | ' -- endsy
| | | | | ' -- semicolon
| | | | | ' -- <subprogram-declaration>
| | | | |   ' -- <function-declaration>
| | | | |     | -- functionsy
| | | | |     | -- ident(IncOne)
| | | | |     | -- <formal-parameter-list>
| | | | |       | -- lparent
| | | | |       | -- <parameter-group>
| | | | |         | | -- <identifier-list>
| | | | |         | |   ' -- ident(x)
| | | | |         | | -- colon
| | | | |         | |   ' -- ident(integer)
| | | | |         | -- rparent
| | | | |     | -- colon
| | | | |     | -- ident(integer)
| | | | |     | -- semicolon
| | | | |     | -- <block>
| | | | |       | -- <declaration-part>
| | | | |       | -- <compound-statement>
| | | | |         | -- beginsy
| | | | |         | -- <statement-list>
| | | | |           | -- <statement>
| | | | |             | | -- <assignment-statement>
| | | | |               | |   | -- <variable>
| | | | |               | |   |   ' -- ident(IncOne)
| | | | |               | |   | -- becomes
| | | | |               | |   | -- <expression>
| | | | |                 | |   | -- <simple-expression>
| | | | |                   | |   | -- <term>
| | | | |                     | |   |   ' -- <factor>
| | | | |                       | |   |     ' -- ident(x)
| | | | |                       | |   | -- <additive-operator>
| | | | |                       | |   |   ' -- plus
| | | | |                       | |   | -- <term>
| | | | |                         | |   |   ' -- <factor>
| | | | |                           | |   |     ' -- intcon(1)
| | | | |                           | |   ' -- semicolon
| | | | |                           | -- endsy
| | | | |                         ' -- semicolon
| | | | | ' -- <compound-statement>
| | | | |   | -- beginsy
| | | | |   | -- <statement-list>
| | | | |     | -- <statement>
| | | | |       | | -- <procedure/function-call>
| | | | |         | |   | -- ident(PrintTwice)
| | | | |         | |   | -- lparent
| | | | |         | |   | -- <parameter-list>
| | | | |         | |   |   ' -- <expression>
| | | | |         | |   |     ' -- <simple-expression>
| | | | |         | |   |       ' -- <term>
| | | | |         | |   |         ' -- <factor>
| | | | |         | |   |           ' -- intcon(2)
| | | | |         | |   ' -- rparent
| | | | |       | -- semicolon
| | | | |     | -- <statement>
| | | | |       | | -- <procedure/function-call>
| | | | |         | |   | -- ident(writeln)
| | | | |         | |   | -- lparent
| | | | |         | |   | -- <parameter-list>
| | | | |         | |   |   ' -- <expression>
| | | | |         | |   |     ' -- <simple-expression>
```

```

|-- <term>
|-- <factor>
|-- <procedure/function-call>
|-- ident(IncOne)
|-- lparent
|-- <parameter-list>
|-- <expression>
|-- <simple-expression>
|-- <term>
|-- <factor>
|-- intcon(5)
|-- rparent
|-- rparent
|-- semicolon
|-- endsy
|-- period
```

Bab 5

Penutup

5.1 Kesimpulan

Milestone 2 berhasil merealisasikan komponen parser untuk Arion Compiler dengan algoritma Recursive Descent yang terstruktur. Parser menerima daftar token dari lexer atau token stream, memverifikasi kesesuaian token dengan grammar Arion, serta membangun parse tree sebagai representasi hierarkis program. Implementasi yang modular memisahkan tanggung jawab antara pembacaan input, pemrosesan token, parsing, dan pencetakan parse tree, sehingga alur kerja mudah ditelusuri.

Hasil pengujian dirancang untuk menunjukkan bahwa parser mampu menangani konstruksi sintaks utama seperti deklarasi, pernyataan kontrol, dan ekspresi, serta melaporkan kesalahan sintaks secara informatif dengan lokasi baris dan kolom. Dengan demikian, parser Milestone 2 telah menyediakan fondasi yang diperlukan untuk melanjutkan ke tahap semantic analysis pada milestone berikutnya.

5.2 Saran

Pengembangan pada milestone selanjutnya dapat memanfaatkan parse tree yang sudah dihasilkan sebagai masukan untuk semantic analysis, termasuk pengecekan tipe, scope, dan konsistensi deklarasi. Selain itu, pengujian dapat diperluas dengan menambahkan variasi kasus yang lebih kompleks, terutama pada struktur subprogram, record, dan array multi-dimensi agar cakupan grammar tervalidasi lebih luas.

Ke depan, kualitas error handling juga dapat ditingkatkan melalui strategi recovery sederhana agar parser mampu melanjutkan parsing setelah menemukan error pertama. Hal ini akan membuat pelaporan error lebih lengkap dalam satu kali eksekusi dan mempermudah proses debugging bagi pengguna.

Bab 6

Lampiran

6.1 Tautan GitHub

Link GitHub

6.2 Tabel Kontribusi

No	Nama Lengkap	NIM	Deskripsi Pekerjaan	Persentase Kontribusi (%)
1	Jonathan Kris Wicaksono	13524023	Merevisi pembacaan separator (spasi, new-line, dan tab)	25
2	Vincent Rionarlie	13524031	Menyusun laporan akhir	25
3	Muhammad Aufar Rizqi Kusuam	13524061	Membuat struktur data keseluruhan untuk Parse Tree	25
4	Bryan Pratama Putra Hendra	13524067	Membuat algoritma core dari parser	25

Bibliografi

- [1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2007.
- [2] Keith D. Cooper and Linda Torczon. *Engineering a Compiler*. Morgan Kaufmann, 2nd edition, 2011.
- [3] Dick Grune and Criel J. H. Jacobs. *Parsing Techniques: A Practical Guide*. Springer, 2nd edition, 2008.
- [4] Dick Grune, Kees van Reeuwijk, Henri E. Bal, Criel J. H. Jacobs, and Koen Langendoen. *Modern Compiler Design*. Springer, 2nd edition, 2012.
- [5] Terence Parr. *Language Implementation Patterns: Create Your Own Domain-Specific and General Programming Languages*. Pragmatic Bookshelf, 2010.
- [6] Seppo Sippu and Eljas Soisalon-Soininen. *Parsing Theory, Volume 1: Languages and Parsing*. Springer, 1990.